Midterm Presentation

Projekt 02: Vision-based Dirt Detection for Cleaning Robot



Object, be cleaned or avoided?

Prepared by： Shufang Tao (3824652), Yuxin Song (3820779), Yuyang Ren (3762594)

Motivation

- Current cleaning robots are already mature in navigation and cleaning execution, but their ability to understand floor objects is still limited.

- In many home environments, cleaning robots still require manual preparation before operation:

  - need to remove chairs, cables, small objects, papers…

  Otherwise,

  - The robot may get stuck and be unable to work
  - or useful things may be treated as garbage



- The robot should not clean mechanically, but make a visual decision:
  - Should this target be cleaned or avoided?

- Our goal is to reduce this preparation effort by enabling vision-based dirt detection. At the same time, improve the robot's flexibility during the cleaning process

Motivation

Commercial robot vacuum	Our project focus
Navigation and coverage planning	Visual dirt/object detection
Obstacle avoidance	Semantic understanding of floor objects
“There is something in front”	“Where is it, and is it dirt or object?”
Cleaning path execution	Dataset-based learning and detection

Camera captures floor image

↓

Image preprocessing

↓

YOLOv3 detection model

↓

Output: dirt / object + bounding box + confidence

↓

Decision layer

- dirt → cleaning target
- object → avoid or ignore

↓

Robot cleaning behavior



Why YOLOv3 for Dirt Detection?

YOLO is used as the object detection framework in this project.

Output: bounding box + class label + confidence
This is what we want!

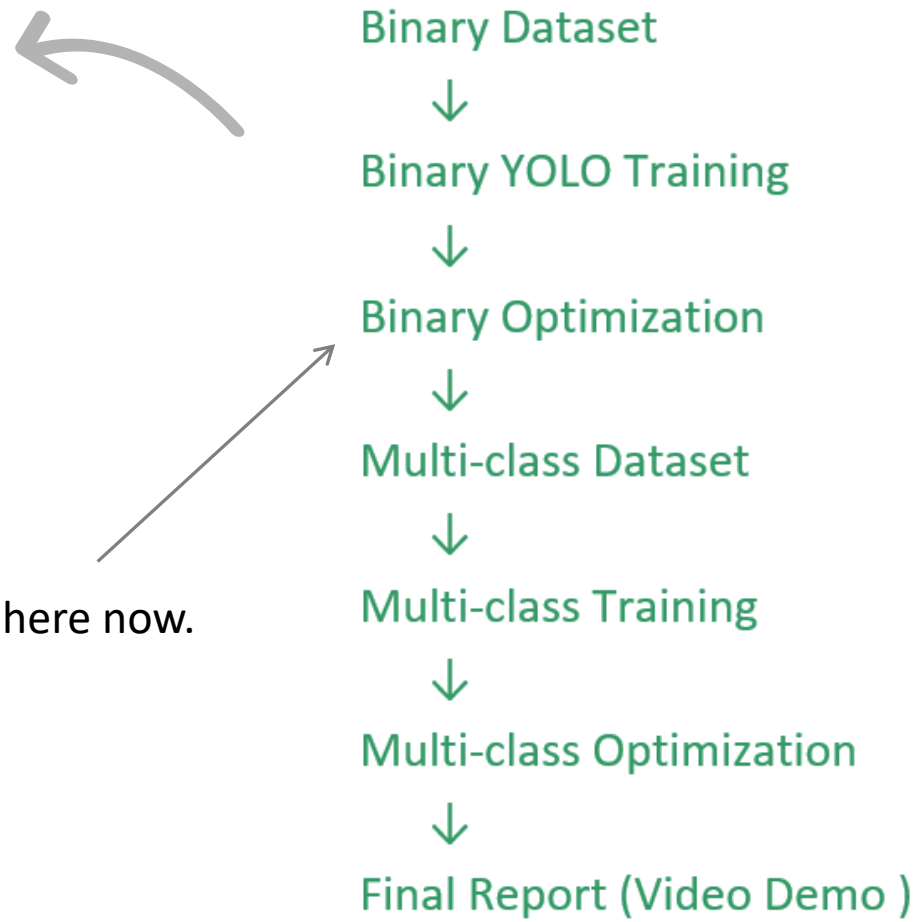


- - One-stage detector: fast inference for robotic applications (Fast, suitable for real-time applications)
- - Object detection instead of image classification: predicts both location and class
- - Multi-scale detection: suitable for small dirt spots and larger objects
- - Transfer learning: initialized from pretrained YOLOv3 weights
- - Mature pipeline: training, validation, inference and mAP evaluation are available
- - Motivated by the DirtNet paper for autonomous cleaning robot scenarios

Planning

- should be cleaned or be avoided :

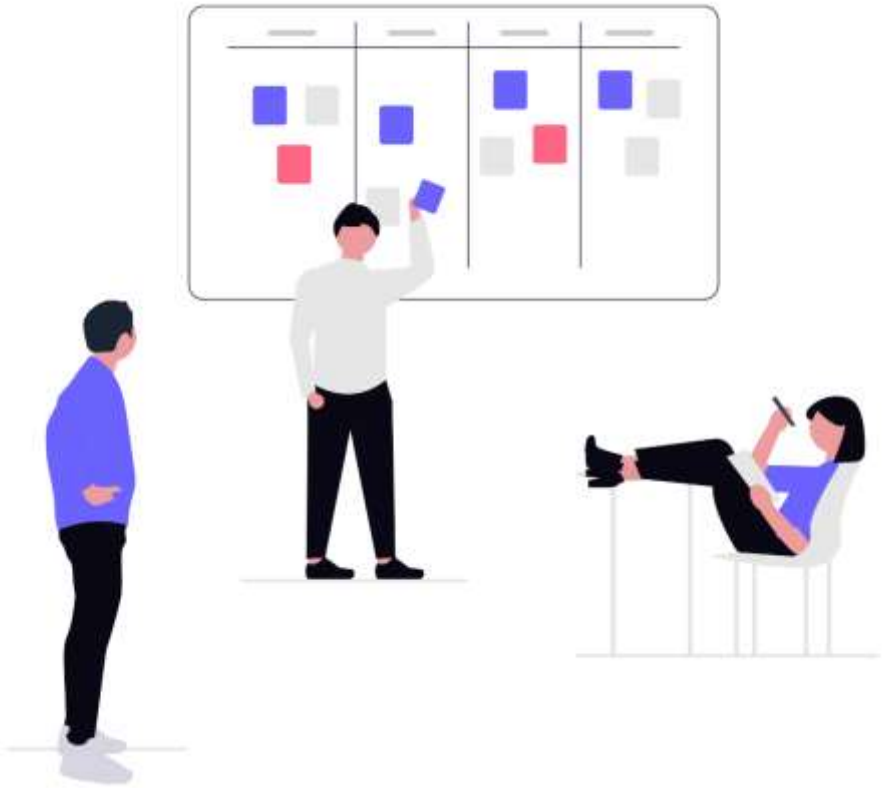
  0 = dirt
  1 = object



Multi-class task:
0 = dirt
1 = pens
2 = paper_and_notebooks
3 = keys
4 = rulers
5 = …
6 = …
7 = …
8 = …
……

Task Assignment

- Member A – Dataset preprocessing:
  - - YOLO binary conversion
  - - train/val/test spli
  - - project planning
- Member B – Training platform:
  - - YOLO environment setup
  - - model training
  - - inference script
- Member C – Evaluation and optimization:
  - - validation and test evaluation
  - - precision / recall / mAP analysis
  - - error case analysis



Dataset preprocessing(YOLO)

```
├── bbox_labels.txt
├── floor_000000_img_000000_r0_f0.png
├── floor_000000_img_000001_r0_f0.png
├── floor_000000_img_000002_r0_f0.png
└── ...
```

convert_to_binary.py

```
data/yolo_binary/
├── images/
│   ├── train/
│   ├── val/
│   └── test/
├── labels/
│   ├── train/
│   ├── val/
│   └── test/
└── dirt_binary.yaml
```

Train_val_1156 （860+296 targets）
test_real_new (80 targets)

- Converted one global annotation file into one .txt label file per image
- Split the dataset into train, validation and test subsets
- Converted bounding boxes from pixel coordinates
  - (xmin, ymin, xmax, ymax, name)
    to YOLO format
    (class_id, x_center, y_center, width, height)

```
floor 000000 img 000000 r0 f0 586 236 640 357 dirt
floor 000000 img 000000 r0 f0 427 498 480 569 dirt
floor 000000 img 000000 r0 f0 380 706 446 783 dirt
floor 000000 img 000000 r0 f0 488 487 534 538 dirt
floor 000000 img 000000 r0 f0 302 611 355 678 dirt
floor 000000 img 000000 r0 f0 102 306 141 332 dirt
floor 000000 img 000000 r0 f0 1073 88 1227 200 pens
floor 000000 img 000000 r0 f0 549 349 809 590 paper_and_notebooks
floor 000000 img 000000 r0 f0 344 745 385 794 keys
floor 000000 img 000000 r0 f0 892 178 1279 518 paper_and_notebooks
```

```
0 0.478906 0.289551 0.042188 0.118164
0 0.354297 0.520996 0.041406 0.069336
0 0.322656 0.727051 0.051562 0.075195
0 0.399219 0.500488 0.035937 0.049805
0 0.256641 0.629395 0.041406 0.065430
0 0.094922 0.311523 0.030469 0.025391
1 0.898438 0.140625 0.120313 0.109375
1 0.530469 0.458496 0.203125 0.235352
1 0.284766 0.751465 0.032031 0.047852
1 0.848047 0.339844 0.302344 0.332031
```

Implementation

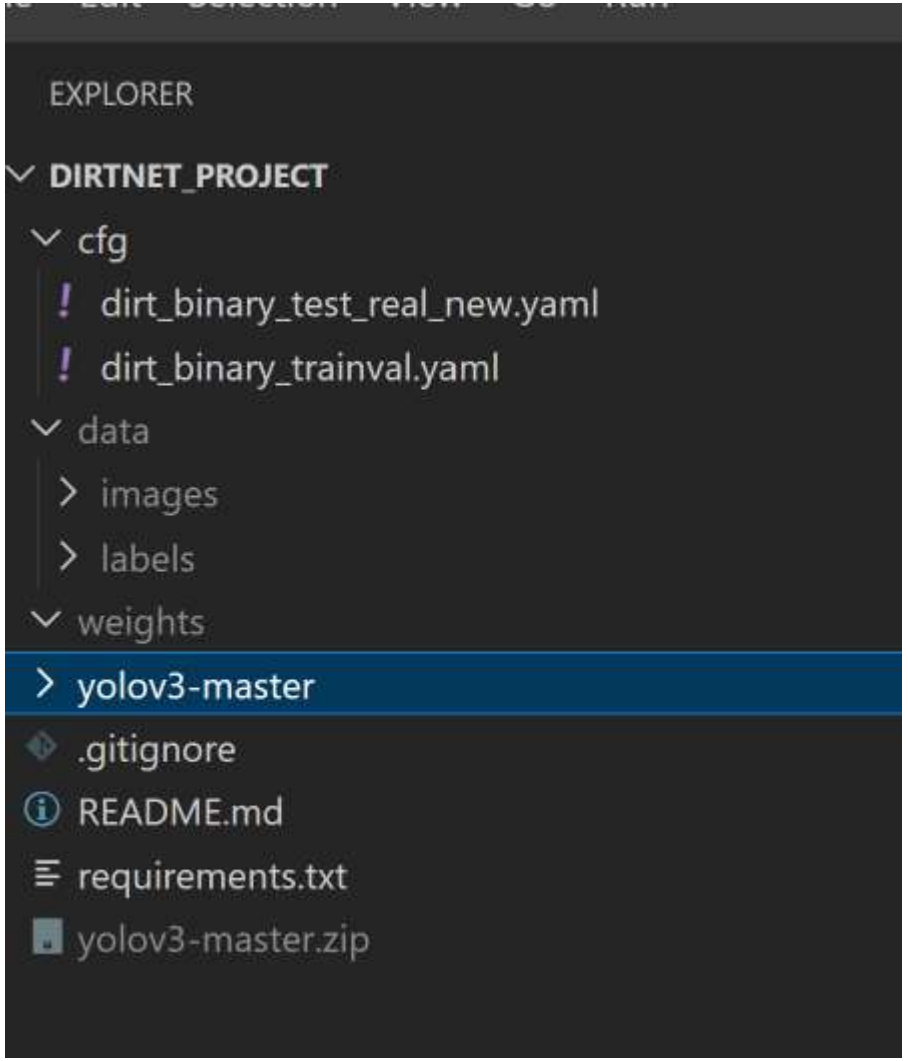
1. Environment & Pipeline Setup

1.Environment ConfigurationCore Dependencies: Configured a stable Anaconda virtual environment using Python 3.8.20, PyTorch 2.4.1, and CUDA 13.0 hardware acceleration on Windows.

2.Optimization: Implemented an optimized 1024x832 rectangular training resolution instead of a standard square input, maintaining high precision while saving 19% in computational cost (FLOPs).

2.Established an Institutionalized Repository Architecture

Built a GitHub repo with code-data decoupling via .gitignore to prevent syncing crashes. Standardized dataset pathways and modular dirt.yaml templates to guarantee absolute environment reproducibility across all team hardware. Finally, authored a comprehensive README.md serving as the team's single source of truth. It details explicit blueprints for virtual environment replication, automated training pipelines, and a standardized one-click deployment protocol to hand over our best.pt weights for real-world inference testing.



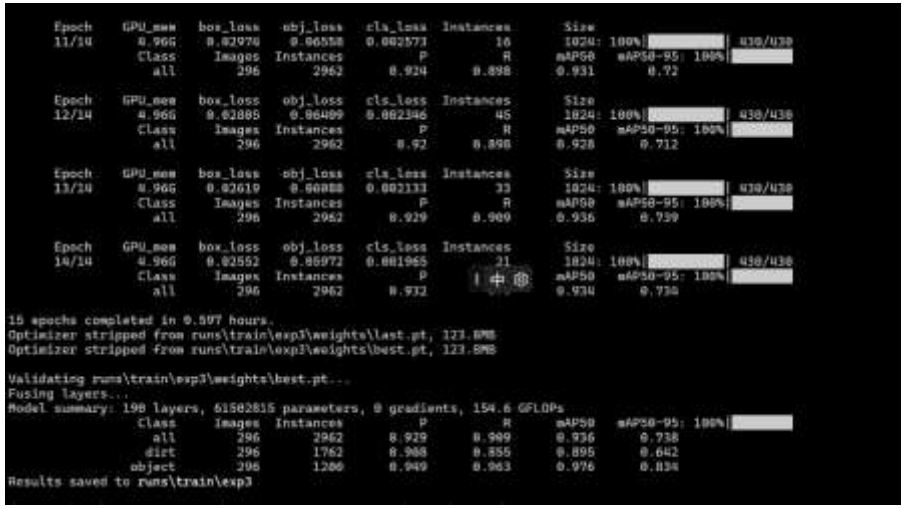
Implementation

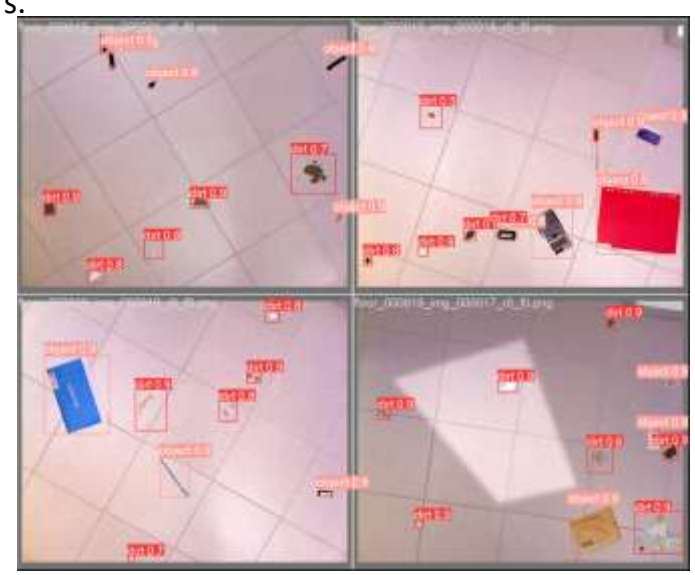
3. Initial Experimental Phase & Evaluation

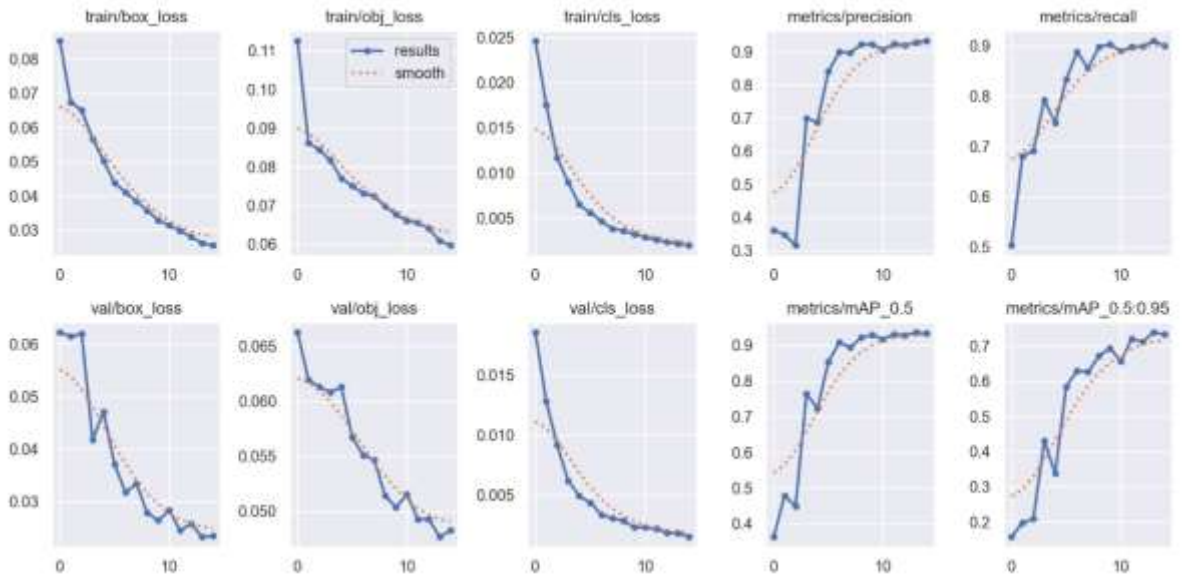
1.First Training Run: Successfully executed the project's first automated training run, completing 15 epochs in approximately 0.6 hours.

2.Strong Baseline Metrics: Achieved an outstanding overall accuracy of 93.6% mAP@0.5 on the first validation set of 296 images, scoring 97.6% on structured objects and 89.5% on dirt.

3.Next Milestone: Formulated the immediate next steps to deploy our best weights (best.pt) against 80 real photographs, evaluating generalization capabilities on actual office floors.



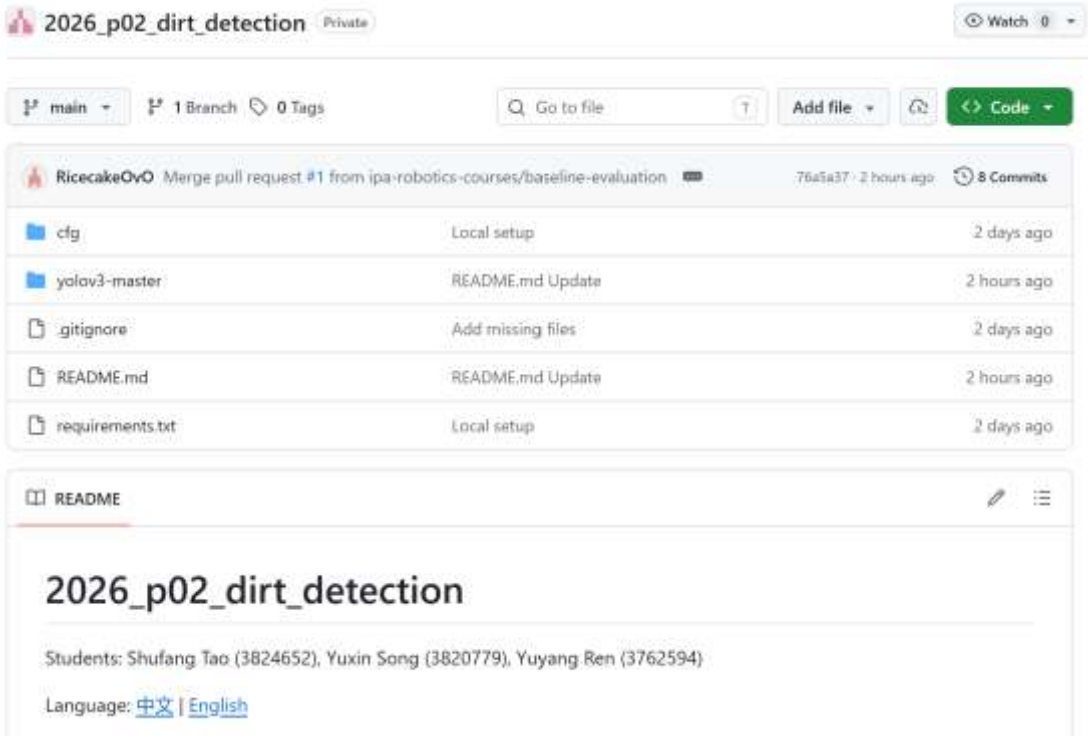




Validation and Testing: Reproducible Workflow Refinement

Confirm the provided baseline can be run and evaluated before interpreting model quality.

- A model result is useful only if it is reproducible
- Updated repository files, README, and dataset configs to align commands, paths, and environment settings with the runnable workflow.
- Checked workflow:
  - README/config files and environment setup
  - Train the baseline model with train.py
  - Run quantitative validation and test-set evaluation with val.py
  - Generate visual detection outputs with detect.py
- Reusable outputs: weights, metric curves, confusion matrix, and prediction labels.



README / code → **Environment** → **Train / Val / Test** → **Baseline results**

Validation and Testing： Evaluation Metrics

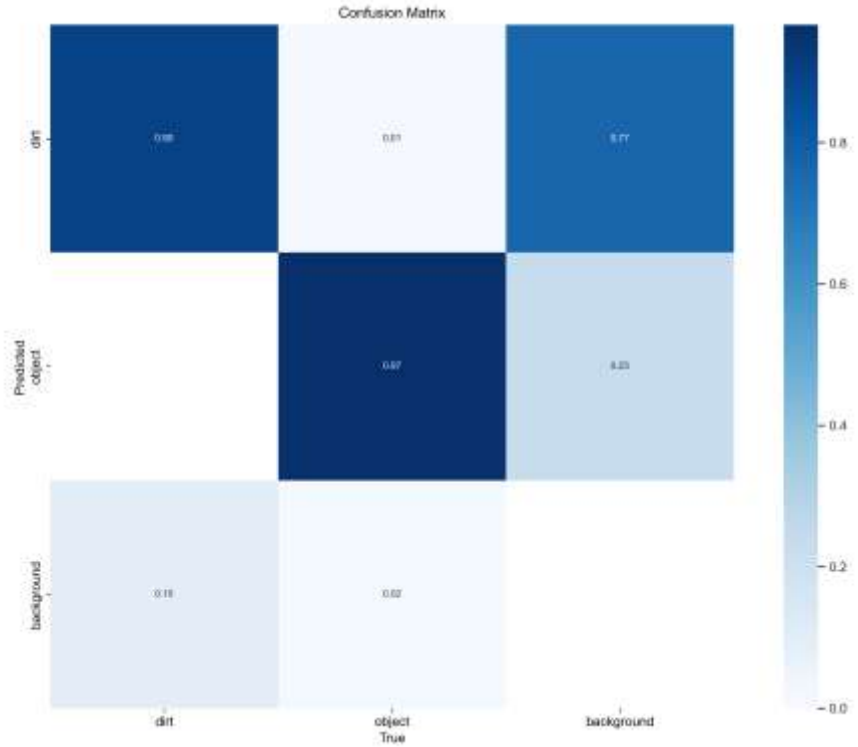
The metrics describe both detection correctness and practical usability.

Metric	Meaning	Value in this project
Precision	Percentage of predicted boxes that are correct	Tells us how reliable the predicted boxes are High precision means fewer false alarms
Recall	Percentage of real targets detected by the model	For cleaning tasks missed dirt may remain uncleaned. High recall means fewer missed dirt regions
F1	Balance between precision and recall F1 = 2 × Precision × Recall / (Precision + Recall)	Useful when P/R trade off A summary score instead of looking at Precision and Recall separately
mAP@0.5	Mean AP(Average Precision) with IoU(Intersection over Union) threshold 0.5	Main object detection metric Evaluates whether predicted boxes match ground-truth boxes with at least 0.5 IoU, and averages performance over classes dirt and object.
Inference time	Time needed for model prediction	Relevant for real-time cleaning use

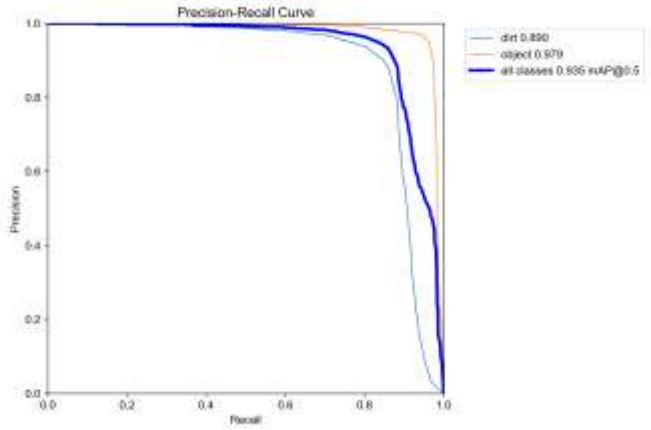
Validation and Testing： Evaluation Result

Validation result from the current 2-class YOLOv3 baseline.

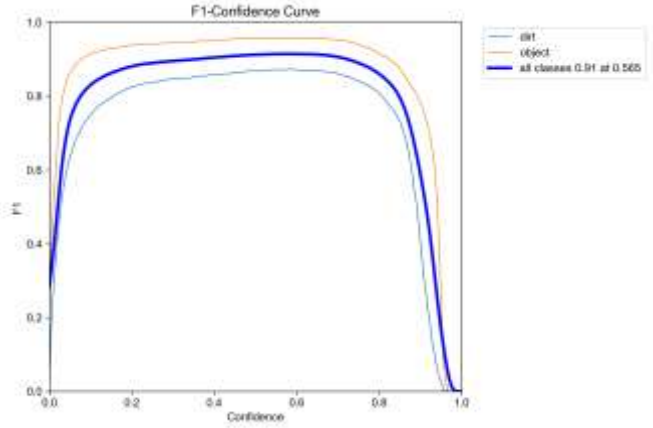
Experiment	Changed parameter	Precision	Recall	mAP@0.5
Baseline	img=1024, epochs=15	**0.9276**	**0.9024**	**0.9344**
Longer training	epochs=30	**0.9395**	**0.9232**	**0.9464**
Smaller image	img=640	**0.8979**	**0.8640**	**0.8973**



Confusion matrix: class errors



PR curve: precision-recall tradeoff



F1 curve: balanced confidence region

Q&A

Thanks for listening.